

ML LAB 1 — WORKFLOW PRATICO: 5 STEP

Guida passo-passo per Regressione Housing

STEP 1: CARICA I DATI

■ **File:** main.py, linea 25

■ **Cosa fa:** Legge dataset California Housing (20k case)

Codice:

```
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing()
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = pd.Series(housing.target * 100000, name='Price')
```

■ **Spiega:**

- fetch_california_housing() scarica da sklearn
- housing.data = features (dimensioni case, età, localizzazione)
- housing.target = prezzi (dividiamo per 100k per leggibilità)
- DataFrame organizza in tabella leggibile

■ **Prova:**

Stampalo: print(X.head())

Quante righe? Quante colonne? (Risposta: 20640 righe, 8 colonne)

STEP 2: DIVIDI IN TRAINING E VALIDATION

■ **File:** main.py, linea 35

■ **Cosa fa:** Split 80% training / 20% test

Codice:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
```

■ **Spiega:**

- test_size=0.2 significa 20% test, 80% training
- random_state=42 rende risultati riproducibili (stesso split ogni volta)
- Training = dati che il modello "vede"
- Test = dati "nuovi" per valutare quanto generalizza

■ **Prova:**

Cambia test_size a 0.1 (solo 10% test, 90% training)

Esegui → vedi MAE cambiare? Il modello migliora o peggiora?

STEP 3: ALLENA IL MODELLO

■ **File:** main.py, linea 45-52

■ **Cosa fa:** Crea 2 modelli e li allena

Modello 1 — Linear Regression:

```
model_lr = LinearRegression()
model_lr.fit(X_train, y_train)
```

- ✓ Semplice: assume relazione lineare ($y = a \cdot x + b$)
- ✓ Veloce da allenare
- ✗ Può underfittare su relazioni complesse

Modello 2 — Random Forest:

```
model_rf = RandomForestRegressor(n_estimators=100, random_state=42)
model_rf.fit(X_train, y_train)
```

- ✓ Complesso: crea 100 alberi di decisione
- ✓ Cattura relazioni non-lineari
- ✗ Più lento, rischio overfitting

■ Spiega:

- `fit(X_train, y_train)` = allena il modello sui dati training
- "Imparare" significa trovare i parametri che riducono l'errore
- Linear Regression: ~10 parametri
- Random Forest: ~1000 parametri (più complesso)

■ Prova:

Commenta RandomForest e aggiungi:

```
from sklearn.ensemble import GradientBoostingRegressor
model_gb = GradientBoostingRegressor()
model_gb.fit(X_train, y_train)
```

È ancora più lento? I risultati migliorano?

STEP 4: VALUTA PERFORMANCE

■ File: main.py, linea 54-62

■ Cosa fa: Misura errore su training e test

Codice:

```
from sklearn.metrics import mean_absolute_error, r2_score

train_mae = mean_absolute_error(y_train, model.predict(X_train))
test_mae = mean_absolute_error(y_test, model.predict(X_test))
test_r2 = r2_score(y_test, model.predict(X_test))
```

■ Spiega le metriche:

MAE (Mean Absolute Error) = errore medio in dollari

- Linear Regression: MAE \$71,234
- Random Forest: MAE \$49,123
- Random Forest sbaglia meno (più accurato)

R² (coefficiente determinazione) = % di varianza spiegata

- Linear Regression: R² 0.58 (spiega 58% della variazione)
- Random Forest: R² 0.76 (spiega 76%)
- R² più alto = modello più bravo

■ OVERFITTING CHECK:

Se `train_mae << test_mae` → overfitting!

Esempio:

Linear: train MAE \$70k, test MAE \$71k ■ BUONO (simili)

Random Forest: train MAE \$10k, test MAE \$50k ■ OVERFITTING (training perfetto, test pessimo)

■ Prova:

Stampa a console per entrambi i modelli:

```
print(f'{name}: train MAE ${train_mae:.0f}, test MAE ${test_mae:.0f}')
```

Quale ha meno differenza? Quale è il modello più robusto?

STEP 5: VISUALIZZA I RISULTATI

■ File: main.py, linea 65-75

■ Cosa fa: Grafico scatter (previsioni vs realtà)

Codice:

```
import matplotlib.pyplot as plt
plt.scatter(y_test, y_pred, alpha=0.3, s=10)
plt.plot([min_val, max_val], [min_val, max_val], 'r--', lw=2)
plt.xlabel('Actual Price ($)')
plt.ylabel('Predicted Price ($)')
plt.show()
```

■ Spiega:

- X-axis: prezzi reali (what actually happened)
- Y-axis: prezzi predetti (what model predicted)
- Linea rossa = perfetto (previsione = realtà)
- Punti sulla linea = modello bravo
- Punti lontani dalla linea = modello sbaglia

Pattern 1: Scatter serrato sulla linea → ■ Buon modello

Pattern 2: Scatter disperso a caso → ■ Cattivo modello

Pattern 3: Scatter con "buco" nel mezzo → ■■ Bias sistematico

■ Prova:

Confronta grafico Linear Regression vs Random Forest

Quale ha scatter più serrato? Quale predice meglio?

ESPERIMENTI PROPOSTI

1) Cambia test_size (0.1, 0.3, 0.5)

→ Come varia MAE?

→ Più test = valutazione più rigorosa, ma meno training

2) Commenta una feature (togli 'HouseAge')

→ R^2 cala molto? Significa che feature era importante

3) Cambia random_state (da 42 a 100)

→ I risultati cambiano? Perché?

→ È importante avere seed fisso per riproducibilità

4) Aggiungi modello nuovo (GradientBoosting, XGBoost)

→ Quale è più veloce?

→ Quale ha R^2 migliore?

TAKEAWAY

■ ML Workflow: carica → split → allena → valuta → visualizza

■ Training MAE vs Test MAE = diagnosi overfitting

■ R^2 alto ma scatter disperso? Check le metriche (potrebbe avere problemi nascosti)

■ Linear semplice, Random Forest complesso — scegli in base al trade-off accuracy/interpretabilità